

Clase 079 — Pivoting y reenvío de puertos

Parte: **3 — Hacking ético y pentesting: metodología** · Fuente: *Offensive Security - "Metasploit Unleashed"; Peter Kim - "The Hacker Playbook 3"* 🕒 Duración estimada: **140 min** · Nivel: **Experto**

Objetivo

El alumno aprenderá a usar un host comprometido como punto de apoyo (pivote) para alcanzar redes internas segmentadas que no son accesibles directamente desde la máquina atacante, dominando túneles SSH, proxychains, el pivoting integrado de Metasploit y herramientas modernas como chisel y ligolo-ng, en un laboratorio propio con topología multi-red.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. Configurar túneles SSH de reenvío local, remoto y dinámico según el escenario de red.
2. Encaminar herramientas de escaneo y explotación a través de un proxy SOCKS con proxychains.
3. Configurar `autoroute` y un proxy SOCKS dentro de Metasploit para pivotar sobre una sesión Meterpreter.
4. Desplegar un túnel con chisel o ligolo-ng cuando SSH no está disponible en el pivote.
5. Diagramar la topología completa de un escenario de doble pivote para un informe de pentest.
6. Explicar por qué la segmentación de red es la contramedida principal frente al pivoting.

Temas

#	Tema	Por qué importa
1	Local port forwarding (<code>ssh -L</code>)	Expone un servicio interno específico en la máquina atacante
2	Remote port forwarding (<code>ssh -R</code>)	Útil cuando el pivote no puede iniciar conexiones salientes hacia el atacante
3	Dynamic forwarding / proxy SOCKS (<code>ssh -D</code>)	Un solo túnel da acceso a toda una subred interna
4	proxychains	Permite encaminar herramientas como nmap a través del pivote
5	Metasploit autoroute + socks_proxy	Pivoting integrado sobre una sesión Meterpreter existente
6	chisel / ligolo-ng	Túneles sobre HTTP(S) cuando SSH no está disponible en el pivote
7	Doble pivote	Encadena dos saltos para alcanzar redes doblemente segmentadas
8	Segmentación como contramedida	Limita el alcance real de un pivote aunque el primer host caiga

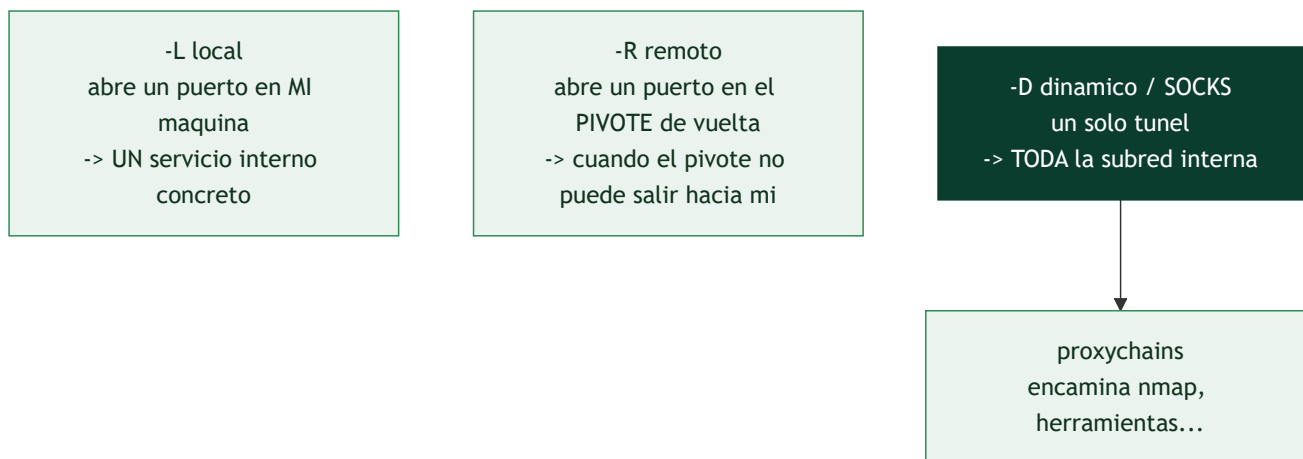
Explicación en profundidad

El host comprometido como puerta a lo inalcanzable

La segmentación de red hace que el atacante no pueda llegar directamente a las zonas sensibles: el servidor de bases de datos vive en una subred a la que su máquina no tiene ruta. El **pivoting** convierte un host ya comprometido —que sí tiene un pie en esa subred— en un **punto** para alcanzarla. No es un ataque nuevo, es una técnica de **enrutamiento**: hacer que el tráfico del atacante viaje *a través* del host pivote hasta destinos que de otro modo serían invisibles. Es la aplicación ofensiva directa de los túneles de la [Clase 037](#).

Los tres reenvíos SSH, y cuándo sirve cada uno

SSH es la herramienta natural del pivoting porque crea túneles sin instalar nada, y sus tres modos responden a tres situaciones distintas:



El **local (-L)** expone en la máquina del atacante un servicio interno **concreto** (el puerto 3306 de un servidor de base de datos interno aparece en `localhost` del atacante). El **remoto (-R)** hace lo contrario y resuelve el caso incómodo: cuando el pivote **no puede iniciar conexiones salientes** hacia el atacante, es el pivote quien abre un puerto de vuelta. El **dinámico (-D)** es el más potente: convierte el cliente SSH en un **proxy SOCKS**, un único túnel a través del cual se alcanza **cualquier** destino que el pivote pueda ver. Combinado con **proxychains**, permite lanzar herramientas como Nmap "a través" del pivote, con los límites ya conocidos (solo TCP con conexión completa: `-sT -Pn`, nada de SYN scan ni UDP).

Cuando SSH no está: Metasploit, chisel y ligolo

No siempre hay un SSH cómodo en el pivote. **Metasploit** integra el pivoting sobre una sesión Meterpreter existente: `autoroute` añade una ruta hacia la subred interna a través de la sesión, y el módulo `socks_proxy` levanta un SOCKS local para que el resto de herramientas la usen —todo sin tocar SSH—. Cuando ni eso encaja, herramientas modernas como **chisel** y **ligolo-ng** crean túneles sobre **HTTP(S)**, lo que les permite atravesar firewalls que solo dejan salir tráfico web y confundirse con navegación normal. Y para redes **doblemente segmentadas** se encadena un **doble pivote**: el primer host da acceso a una subred, y un segundo host comprometido en ella da acceso a una tercera. Cada salto añade latencia y fragilidad, así que se hace con intención, no por deporte.

El reverso defensivo: por qué la segmentación funciona

Toda esta clase es, leída al revés, un argumento a favor de la **segmentación de red** como contramedida. El pivoting solo tiene sentido porque hay una red interna que alcanzar; si esa red está bien segmentada —con firewalls internos que restringen qué host puede hablar con cuál, no solo el perímetro— la caída del primer host no entrega la red entera, sino solo lo que ese host podía ver legítimamente. El movimiento **este-oeste** (entre hosts internos) es el que el pivoting explota y el que la microsegmentación de la [Clase 042](#) restringe. Documentar en el informe **qué se alcanzó mediante pivoting y desde dónde** es lo que permite al cliente ver el valor de segmentar: no es teoría, es "desde este servidor web comprometido llegué a vuestra base de datos de nóminas porque no había nada entre ellos".

Definiciones y características

- **Pivoting**: técnica de usar un host comprometido como intermediario para acceder a redes que no son directamente alcanzables desde la máquina del atacante. Característica clave: el tráfico del atacante viaja encapsulado dentro de una conexión legítima ya establecida con el pivote.

- *Enfoque defensivo*: la segmentación de red real (VLANs, firewalls internos, zero trust) es la única contramedida que limita el alcance de un pivote incluso si el primer host cae.
- **Local port forwarding (-L)**: mapea un puerto de la máquina atacante a un destino alcanzable desde el pivote. Característica clave: `ssh -L 8080:172.16.0.10:80 usuario@pivote` hace que `localhost:8080` en el atacante equivalga al puerto 80 del host interno.
- *Enfoque defensivo*: monitorizar sesiones SSH de larga duración con reenvío de puertos activo (visible en `netstat` /EDR) ayuda a detectar túneles no autorizados.
- **Remote port forwarding (-R)**: abre un puerto en el pivote que reenvía las conexiones hacia la máquina atacante. Característica clave: útil cuando el firewall del pivote bloquea conexiones salientes hacia el atacante pero sí permite conexiones entrantes locales.
- *Enfoque defensivo*: restringir puertos abiertos localmente en servidores críticos y auditar reglas de firewall saliente reduce las opciones disponibles para este tipo de túnel.
- **Dynamic port forwarding / SOCKS (-D)**: convierte el túnel SSH en un proxy SOCKS genérico. Característica clave: un único túnel dinámico da acceso a cualquier host y puerto de la subred interna, sin necesitar un mapeo por servicio.
- *Enfoque defensivo*: un solo túnel SOCKS activo puede exponer una subred completa; por eso los servidores bastión deben limitar estrictamente qué usuarios pueden abrir sesiones SSH interactivas.
- **proxychains**: utilidad que fuerza a cualquier programa a enrutar su tráfico TCP a través de un proxy SOCKS/HTTP configurado. Característica clave: solo soporta conexiones TCP completas, por lo que escaneos SYN de nmap no funcionan a través de él.
- *Enfoque defensivo*: el tráfico proxiado suele generar patrones de conexión inusuales (mismo origen, múltiples destinos internos en poco tiempo) detectables por un NDR bien calibrado.
- **chisel**: herramienta que crea túneles TCP/UDP encapsulados sobre HTTP, útil cuando SSH no está disponible en el pivote (por ejemplo, un host Windows). Característica clave: puede operar en modo `--reverse`, iniciando la conexión desde el pivote hacia el atacante, lo cual evade restricciones de firewall saliente estrictas.
- *Enfoque defensivo*: inspeccionar tráfico HTTP/HTTPS saliente con un proxy corporativo que valide certificados y patrones de tráfico puede exponer túneles chisel disfrazados de tráfico web normal.
- **ligolo-ng**: herramienta moderna de pivoting que crea una interfaz de red virtual (TUN) en la máquina atacante, en lugar de depender de proxychains. Característica clave: al operar a nivel de interfaz de red, permite usar cualquier herramienta (incluyendo escaneos SYN) sin las limitaciones de un proxy SOCKS tradicional.
- *Enfoque defensivo*: la detección de agentes ligolo-ng ejecutándose en un host (proceso desconocido con conexión saliente persistente) es un buen caso de uso para reglas de EDR basadas en comportamiento.

Glosario

Término	Definición concisa
Pivoting	Usar un host comprometido para alcanzar redes inaccesibles
Host pivote	Máquina con acceso a la subred objetivo
Local forwarding (-L)	Expone un servicio interno concreto en el atacante
Remote forwarding (-R)	Puerto de vuelta cuando el pivote no puede salir
Dynamic forwarding (-D)	Proxy SOCKS: acceso a toda la subred por un túnel
SOCKS	Proxy genérico que transporta TCP con conexión completa
proxychains	Encamina herramientas a través del proxy del túnel
autoroute	Ruta de Metasploit hacia una subred vía Meterpreter
socks_proxy	Módulo de Metasploit que levanta un SOCKS local
chisel / ligolo-ng	Túneles sobre HTTP(S) cuando no hay SSH
Doble pivote	Encadenar dos saltos para redes doblemente segmentadas
Tráfico este-oeste	Comunicación entre hosts internos
Segmentación	Contramedida que limita el alcance de un pivote
Latencia por salto	Cada pivote añade retardo y fragilidad

Herramientas y preparación

- **Entorno de laboratorio:** una topología con al menos tres redes virtuales: la red del atacante (Kali), una red intermedia con el host pivote comprometido, y una red interna aislada solo alcanzable desde el pivote. Se monta con VirtualBox/VMware usando redes internas ("Internal Network") sin acceso directo desde el host atacante.
- **Cliente SSH:** incluido en Kali por defecto; el pivote debe tener acceso SSH ya comprometido de clases previas.
- **proxychains4:** `sudo apt install proxychains4`; configurar `/etc/proxychains4.conf` con la línea `socks5 127.0.0.1 1080` (o el puerto elegido).
- **chisel:** binario único, se descarga para Linux y Windows desde su release de GitHub; no requiere instalación, solo transferirlo al pivote.
- **ligolo-ng:** requiere un binario "proxy" en la atacante y un binario "agent" en el pivote, además de crear una interfaz TUN con privilegios de administrador en la máquina atacante.
- **Metasploit Framework:** para el módulo `post/multi/manage/autoroute` y `auxiliary/server/socks_proxy`, sobre una sesión Meterpreter ya activa.
- **nmap:** para validar el alcance de la red interna antes y después de establecer el pivote, comparando resultados.
- **Wireshark:** opcional, útil para observar cómo se ve el tráfico encapsulado de cada tipo de túnel en el laboratorio.

El pivoting extiende deliberadamente tu acceso hacia redes que suelen estar segmentadas por diseño de seguridad. Practica esto solo dentro de tu laboratorio propio o del alcance explícitamente autorizado en las reglas de engagement de un pentest contratado. Alcanzar una red no incluida en el alcance firmado es una violación grave del contrato y puede constituir delito.

1. Con acceso SSH al pivote (`10.10.10.5`) y una subred interna objetivo `172.16.0.0/24` , levanta un túnel dinámico: `ssh -D 1080 -N usuario@10.10.10.5` .
2. Escanea la red interna a través de proxychains: `proxychains4 nmap -sT -Pn -p 22,80,445 172.16.0.10` .
3. Expón un servicio interno específico con local forwarding: `ssh -L 8080:172.16.0.10:80 -N usuario@10.10.10.5` y verifica con `curl http://127.0.0.1:8080` .
4. Sobre una sesión Meterpreter activa en el pivote, configura autoroute desde Metasploit: `use post/multi/manage/autoroute` , `set SESSION 1` , `set SUBNET 172.16.0.0` , `run` .
5. Levanta el proxy SOCKS integrado de Metasploit: `use auxiliary/server/socks_proxy` , `set SRVPORT 1080` , `set VERSION 4a` , `run -j` .
6. Si el pivote no tiene SSH disponible (por ejemplo, es Windows sin cliente SSH), levanta un túnel con chisel: en Kali, `./chisel server -p 8000 --reverse` ; en el pivote, `.\chisel.exe client 10.10.10.5:8000 R:socks` .
7. Configura proxychains para apuntar al puerto SOCKS de chisel o de Metasploit y repite el escaneo de la red interna.
8. Como alternativa moderna, despliega ligolo-ng: ejecuta el `proxy` en Kali, crea la interfaz con `ip tuntap add user root mode tun ligolo` , súbela con `ip link set ligolo up` , ejecuta el `agent` en el pivote y añade la ruta hacia la subred interna con `ip route add 172.16.0.0/24 dev ligolo` .
9. Con la interfaz ligolo-ng activa, ejecuta un escaneo SYN directo (`nmap -sS 172.16.0.0/24`) y compáralo contra el escaneo `-sT` realizado antes por proxychains.
10. Captura tráfico con Wireshark en el segmento del pivote durante uno de los túneles para observar cómo se ve encapsulado el tráfico interno.
11. Diagrama la topología completa: atacante → pivote → subred interna, señalando cada túnel usado y el puerto correspondiente.

Ejercicios

1. Explica con un ejemplo concreto la diferencia entre `-L` , `-R` y `-D` en un túnel SSH.
2. Escanea un host de la subred interna con proxychains y nmap, y justifica por qué debes usar `-sT -Pn` en lugar de un escaneo SYN.
3. Configura `autoroute` en Metasploit y verifica la ruta activa con el comando `route print` dentro de msfconsole.
4. Levanta un proxy SOCKS con chisel en modo `--reverse` y encamina una petición `curl` a través de él usando proxychains.
5. Diseña (en diagrama) un escenario de doble pivote de dos saltos hacia un objetivo final, indicando qué herramienta usarías en cada salto y por qué.
6. Compara ligolo-ng frente a proxychains y explica en qué escenario elegirías cada una.

Reto verificable

Reto: alcanza un servicio en la subred interna de tu laboratorio que no es accesible directamente desde Kali, exclusivamente a través de un pivote, usando al menos dos técnicas distintas (por ejemplo, SSH dinámico y chisel, o autoroute y ligolo-ng).

Criterio de aceptación: el alumno demuestra acceso al servicio interno (respuesta HTTP, banner de servicio o resultado de escaneo) inalcanzable sin el túnel, documenta el método exacto usado en cada caso y adjunta un diagrama de la topología atacante-pivote-objetivo.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
nmap con SYN scan falla a través de proxychains	proxychains solo soporta conexiones TCP completas (connect); usar siempre <code>-sT -Pn</code>
El proxy SOCKS no responde	El túnel se cayó o el puerto configurado en <code>proxychains4.conf</code> no coincide con el del túnel activo; verificar el listener con <code>netstat</code>
autoroute no encamina el tráfico	La <code>SESSION</code> especificada es incorrecta o la subred está mal escrita; revisar con <code>route print</code> en msfconsole
chisel no logra conectar	Firewall saliente bloqueando el puerto; usar <code>--reverse</code> y un puerto comúnmente permitido como 443
ligolo-ng no crea la interfaz TUN	Falta el privilegio de administrador/root en la máquina atacante para crear la interfaz de red
Rendimiento muy bajo en doble pivote	Latencia acumulada de dos túneles anidados; reducir el número de puertos escaneados y evitar paralelismo excesivo

Preguntas frecuentes

 **¿Por qué un escaneo nmap con proxychains necesita las banderas `-sT` y `-Pn`?** Porque proxychains solo puede encaminar conexiones TCP de tipo connect (establecidas a nivel de aplicación), no paquetes SYN crudos; además, el ping ICMP previo tampoco se transmite por el proxy, así que hay que omitirlo con `-Pn`.

 **¿Cuándo conviene usar remote forwarding (`-R`) en lugar de local (`-L`)?** Cuando el pivote tiene un firewall saliente restrictivo que impide iniciar conexiones nuevas hacia el atacante, pero sí permite exponer un puerto local que reenvíe hacia una escucha que el atacante ya preparó.

 **¿Chisel o un túnel SSH tradicional?** SSH es la opción natural si el pivote ya lo tiene disponible y es Linux. Chisel es preferible cuando no hay cliente SSH (por ejemplo, un pivote Windows), cuando solo se permite tráfico HTTP/HTTPS saliente, o cuando se necesita atravesar un proxy corporativo.

 **¿Cómo detecta un equipo defensivo el pivoting?** Mediante segmentación real de red con firewalls internos, sistemas de detección de tráfico anómalo entre segmentos (NDR), monitoreo de conexiones salientes inusuales desde servidores internos, y auditoría de túneles SSH o procesos desconocidos como chisel ejecutándose en hosts críticos.

? ¿ligolo-ng reemplaza completamente a proxychains? En muchos escenarios sí, porque al trabajar a nivel de interfaz de red permite usar cualquier herramienta sin las limitaciones de un proxy SOCKS; sin embargo, proxychains sigue siendo útil cuando no se puede o no se quiere crear una interfaz TUN en la máquina atacante.

Referencias

- Offensive Security, *Metasploit Unleashed* - Pivoting — <https://www.offsec.com/metasploit-unleashed/pivoting/>
- Peter Kim, *The Hacker Playbook 3*, Secure Planet LLC.
- chisel (jpillora) — <https://github.com/jpillora/chisel>
- ligolo-ng (Nicolas Chatelain) — <https://github.com/nicocha30/ligolo-ng>
- MITRE ATT&CK, T1090 Proxy — <https://attack.mitre.org/techniques/T1090/>

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Clase anterior

[Clase 078](#) — Movimiento lateral en la red

Siguiente clase

[Clase 080](#) — Cracking de contraseñas con John y Hashcat